Neural Network Function Approximation

Anastasia Istratuca
Supervisor: Dr. Daniel Loghin

1 Abstract

This project studies a novel function approximation method and its applications. We consider both vastly studied algorithms, such as polynomial or rational approximation, however the emphasis is on the neural network approximation routine. The goal is to illustrate the potential of this technique to anyone with a good understanding of linear algebra without requiring any prior knowledge on approximation theory or machine learning. Special attention is given to the aspects of problem formulation and mathematical analysis which lead to an approximation algorithm suitable for implementation on a computer program.

2 Introduction

Approximation theory is the branch of mathematics concerned with how functions can be approximated by simpler, more suitable functions. In other words, given $f : [a, b] \to \mathbb{R}$, find $g : [a, b] \to \mathbb{R}$ such that, for some small $e : [a, b] \to \mathbb{R}$:

$$f(x) \approx g(x) \iff f(x) = g(x) + e(x), \quad \forall x \in [a, b]. \tag{1}$$

There are two principal uses of function approximation. Firstly, replacing a complicated looking function by a simpler one, many common operations such as differentiation, integration and even evaluation become easier to perform. For example, $\ln(x)$, $\sin(x)$ or the error function cannot be evaluated using basic arithmetic operations without approximating them first. Secondly, suppose the values of the function $f(x)$ are known only for a finite, typically small, number of points $\{x_i\}_{i=0}^n$. Function approximation allows us to generate values of $f(x)$, for all points x in the approximating interval, where $x \neq x_i$, $i = 0, \ldots, n$. This is also known as data-fitting.

The study of approximation theory covers two general types of problems, corresponding to the two applications above. In the first problem, the function f is given explicitly and we wish to replace it with a simpler function g. The second problem involves fitting functions to given data. In other words, instead of an explicit formula, only a set of points $\{(x_i, f(x_i)\}_{i=0}^n$ is provided and we seek to find the best function g to represent it.

The most common approach to finding the approximating function g consists of first choosing a suitable space S such that $g \in S$, and a basis of S, $\{g_i\}_{i=0}^n$. Then, since $g \in S$, g can be expressed as a linear combination of $\{g_i\}_{i=0}^n$, so that:

$$g(x) = \sum_{i=0}^{n} a_i g_i(x), \tag{2}$$

for some $a_i \in \mathbb{R}$, $i = 0, \ldots, n$. Our problem reduces to finding $\{a_i\}_{i=0}^n$ so that (1) is satisfied.

Most procedures in approximation theory are numerical methods, namely algorithms which only use arithmetic operations and some logical operations, such as algebraic comparison. An equally important notion is the error introduced in approximating the solution of a mathematical problem by a numerical method. The error is particularly useful for measuring the quality of the approximation, hence for comparing two distinct approximating techniques. We will mostly investigate the so-called L^∞ error, defined as:

$$\|f - g\|_\infty := \max_{x \in [a,b]} |f(x) - g(x)|.$$

3 Three simple approximating functions

3.1 Polynomial Functions

Perhaps the most common space S considered is the set of polynomials of degree n, denoted as Π_n. In this case, the problem we are aiming to solve can be formulated as follows:

$$\text{Given } f : [a, b] \to \mathbb{R}, \text{ find } p \in \Pi_n \text{ such that } f(x) \approx p(x), \forall x \in [a, b].$$

The natural question now is, given an arbitrary function f, does the polynomial p exist? And if so, can we always find p?

Theorem 3.1 (*Weierstrass Approximation Theorem* (Cheney, 1966))**.** Let f be a continuous function defined on $[a, b]$. Given $\epsilon > 0$, there exists a polynomial P such that $\|f - P\|_\infty < \epsilon$.

Weierstrass' Theorem confirms that there exists a polynomial p that approximates f with arbitrarily high precision. However, it does not provide an algorithm for finding p or any additional information about it, such as the degree of the polynomial p.

Polynomials are not only used in function approximation, but also in approximating the roots of a polynomial expression of any degree, in approximating derivatives, integrals and solutions to both ordinary and partial differential equations. They are the basic means in almost all areas of numerical analysis, mainly due to their simple structure, which makes them easy to manipulate. Furthermore, the space Π_n is convenient for modelling problems, such as function approximation, because polynomials have well understood properties. For instance, they are continuous on $\mathbb{R}$ and infinitely differentiable. They have fairly good flexibility and "curve-fitting" power, and they are computationally easy to use.

However, there are also some disadvantages to consider. First of all, polynomials have poor extrapolatory properties. That is, even though they provide a good fit inside the approximation interval, their performance tends to fail rapidly outside the range of data. Additionally, they often generate highly unstable models, or in other words, models where small changes in the input x, for some $x \in [a, b]$, lead to large changes in the output $p_n(x)$.

3.2 Trigonometric Approximation

Another useful space S considered in approximation theory is T_n, the space of trigonometric polynomials of degree n. Its most common basis is $\{\sin(kx), \cos(kx)\}_{k=0}^n$, also known as Fourier functions. T_n is particularly convenient for describing functions f which represent physical phenomena that have periodic behaviour, such as light and sound. For this reason, the function f is chosen to be periodic too, that is:

$$f(x + \tau) = f(x).$$

Since trigonometric functions are periodic as well, it is intuitive to choose an approximating function from the space T_n. Note that $\sin$ and $\cos$ are preferred to $\tan$ and $\cot$, as the former are defined everywhere on $\mathbb{R}$. For this reason, we define the trigonometric polynomial (Atkinson, 1989):

$$p(x) = \sum_{k=0}^{n} (a_k \cos(kx) + b_k \sin(kx))$$

If, in addition, $|a_n| + |b_n| \neq 0$, p_n is called a trigonometric polynomial of degree n. Having defined the functions in T_n, we can now state the problem formulation:

$$\text{Given } f : [a, b] \to \mathbb{R}, \text{ find } p \in \mathrm{T}_n \text{ such that } f(x) \approx p(x), \forall x \in [a, b],$$

Apart from their periodic behaviour, $\sin$ and $\cos$ also have some of the advantages that polynomials exhibit: they are continuous and infinitely differentiable on $\mathbb{R}$, they have moderate flexibility

and they are not extremely computationally expensive. Moreover, $\sin$ and $\cos$ are often used in the form of Fourier Series and they are especially helpful for approximating solutions to various ordinary and partial differential equations.

However, the space T_n is generally applicable for obtaining an approximation for periodic functions. Since we are interested in approximating any continuous function f, not necessarily periodic, we now introduce a different, more general space S for deriving the approximating function g.

3.3 Rational Approximation

We have seen in Section 3.1 that, on the one hand, polynomials have distinct advantages for usage in approximation. On the other hand, they are unstable and tend to oscillate heavily outside the approximation interval. We would like to overcome these disadvantages, while still making use of the approximation power of polynomials. For this reason, we define another type of approximating function g given by the ratio of two polynomials, as follows:

$$g(x) = \frac{\sum_{i=0}^{n} a_i x^i}{\sum_{j=0}^{m} b_j x^j} = \frac{p_n(x)}{p_m(x)}, \quad p_n \in \Pi_n, p_m \in \Pi_m,\ a_i, b_i \in \mathbb{R},\ i = 0, \ldots, n,\ j = 0, \ldots, m,\ n > m.$$

This is known as rational approximation. We use $\mathrm{R}_{n,m}$ to denote the space of ratios of two polynomials of degrees n and m respectively, with $n > m$. However, note that g is no longer a linear combination as in equation 2, which is a key difference between the approximation considered in Section 3.1 and the approximation considered in this section. Then the problem we seek to solve can be formulated as follows:

$$\text{Given } f : [a, b] \to \mathbb{R}, \text{ find } q \in \mathrm{R}_{n,m} \text{ such that } f(x) \approx q(x),\ \forall x \in [a, b],$$

Thus, given an arbitrary function f, does the ratio q exist? To answer this question, we first define a subspace of $\mathrm{R}_{n,m}$, namely the space of irreducible ratios on $[a, b]$, $\overline{\mathrm{R}}_{n,m}[a, b]$, as follows:

$$\overline{\mathrm{R}}_{n,m}[a, b] = \left\{ \frac{p_n(x)}{p_m(x)} \middle| p_n \in \Pi_n;\ p_m \in \Pi_m;\ p_m(x) > 0,\ \forall x \in [a, b];\ \text{gcd}(p_n, p_m) = 1 \right\},$$

where $\text{gcd}(p_n, p_m)$ denotes the greatest common divisor of the polynomials p_n and p_m.

Theorem 3.2 (*Existence of Rational Approximation* (Cheney, 1966))**.** Let f be a continuous function defined on $[a, b]$. Given $\epsilon > 0$, there exists an irreducible ratio $Q \in \overline{\mathrm{R}}_{n,m}[a, b]$ such that $\|f - Q\|_\infty < \epsilon$.

Rational approximation has a few advantages over polynomial approximation. First of all, ratios are more flexible than polynomials, particularly, they can adapt to a wider range of shapes. They are typically smoother and less oscillatory, and they behave well outside the approximation interval. Moreover, Cheney (ibid.) states that a rational $q \in \mathrm{R}_{n,m}$ has roughly the same approximation or "curve-fitting" power as a polynomial $p \in \Pi_{n+m}$. In other words, ratios provide the same approximation power, but with a lower polynomial degree both in the numerator and denominator.

Nevertheless, rational approximation is not as widely-used as the polynomial model. On the one hand, that is because the properties of rational functions are not as well understood as those of polynomials. In particular, we cannot yet relate the degrees in the numerator and denominator with the given data or with the approximation error. On the other hand, if we do not impose any constraints, the approximating ratio can sometimes result in vertical asymptotes as we do not explicitly impose that the denominator has no roots in the definition of $\mathrm{R}_{n,m}$.

Even though ratios indeed represent a non-linear model, they are a particularly easy one to fit, especially from a computational point of view. Still, an important remark that rational approximation illustrates is that non-linear approximations appear to be superior to the linear ones.

4 Neural Network Approximation

In the previous section, we introduced a model which was non-linear in its coefficients. We now explore models with a different type on non-linearity. Much like in the trigonometric approach, we implement a linear combination of non-linear functions. Though, we do not restrict ourselves to periodic models.

Consider the following function:

$$g : \mathbb{R} \to \mathbb{R},\ g(x) = \sum_{i=0}^{n} a_i \phi_i(x)$$

We set $\phi_0 = 1$, so that our approximating function becomes:

$$g(x) = a_0 + \sum_{i=1}^{n} a_i \phi_i(x) \tag{3}$$

The function g defines, in fact, a particular type of neural network, namely a shallow neural network, where the parameters a_i are denoted by w_i and represent the weights of the network, a_0 is generally denoted by b and represents the bias, and ϕ is the corresponding activation function.

Although representation (3) gives a mathematical description of a neural network with one hidden layer, we now continue this section with defining the notion of artificial neural network and its components. To better understand this, consider Figure 1.

4.1 Artificial Neural Networks

Formally, an Artificial Neural Network is a computational model based on the human nervous system, in particular on the human brain. Such systems perform tasks by learning from different examples. For instance, in the field of image recognition, a neural network can be trained to identify cars by looking at different images labelled as 'cars' or 'no cars'.

Each connected unit of the network is called a neuron, which similarly models the biological neuron. Neurons are further clustered into layers, where the first layer is called the input layer, the last layer is called the output layer and the middle layers are called hidden layers. If a neural network has only one hidden layer, then it is called a Shallow Neural Network, while if the number of hidden layers is greater than one, it is called a Deep Neural Network.

The neurons are connected by edges, more commonly known as weights, and each neuron has a bias. The weights and biases of a network are real numbers organised in matrix or vector form. Suppose we have a neural network with L layers and that each layer has n_i neurons. Then, n_1 is the number of neurons in the input layer, which also represents the dimension of the input, and n_L is the number of neurons in the output later, or equivalently, the dimension of the output.

Therefore, a neural network is a function, say g, such that $g : \mathbb{R}^{n_1} \to \mathbb{R}^{n_L}$. For each layer i, $i = 2 \dots L$, w_i denotes the matrix of weights, so that $w_i \in \mathbb{R}^{n_i \times n_{i-1}}$, and b_i is the vector of biases,

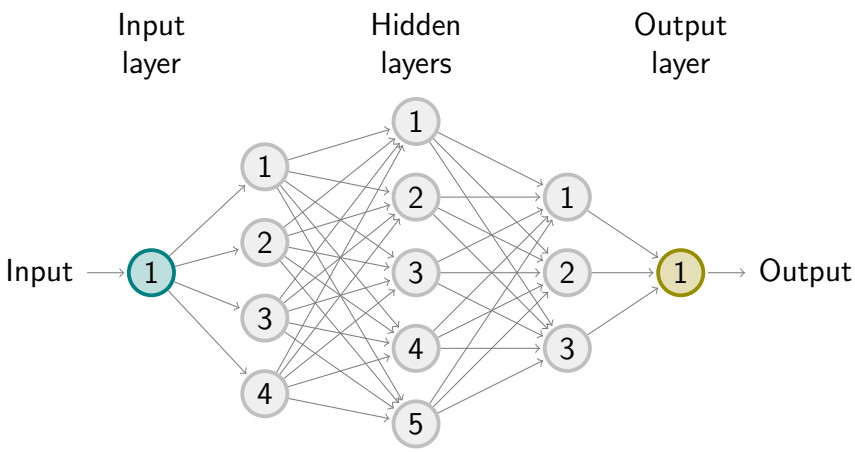


Figure 1: Deep Neural Network with 5 layers.

so that $b_i \in \mathbb{R}^{n_i}$. Note that this is in accordance with (3) for shallow neural networks with $n := n_2$ neurons in the hidden layer, $n_1 = 1$, and $n_3 = 1$. Thus, in this case, the total number of parameters of the network is $n \times 1 + 1 \times n = 2n$ weights and $n+1$ biases, in total $3n+1$ parameters. Moreover, $n_1 = 1$ indicates that the function we aim to approximate is a function of one variable. This further implies that if we seek to approximate a function of d variables, where $d \in \mathbb{N}$, then we require $n_1 = d$ and $n_L = 1$, if the output of the investigated function is a real number.

Further, we define the weighted input at layer i by:

$$z_i = w_i \phi_{i-1} + b_i, \quad z_i \in \mathbb{R}^{n_i}, \quad 2 \leqslant i \leqslant L,$$

and the output or activation at layer i by:

$$\phi_i = \phi(z_i), \quad a_i \in \mathbb{R}^{n_i}, \quad 1 \leqslant i \leqslant L-1 \text{ and } \phi_1 = x \in \mathbb{R}^{n_1}.$$

The last and most important component of a neural network is the activation function, which is used to define the output of a certain layer. Commonly used activation functions are the Rectified Linear Unit, given by $\phi(x) = \max\{0, x\}$ and the logistic sigmoid activation function $\phi(x) = \frac{1}{1+e^{-x}}$.

To better understand this, in Figure 1, we provide an example of a neural network with 5 layers (3 hidden layers), hence $L = 5$. In this particular example, we have:

$$n_1 = 1,\ n_2 = 4,\ n_3 = 5,\ n_4 = 3,\ n_5 = 1;$$
$$w_2 \in \mathbb{R}^{4\times 1},\ w_3 \in \mathbb{R}^{5\times 4},\ w_4 \in \mathbb{R}^{3\times 5},\ w_5 \in \mathbb{R}^{1\times 3};$$
$$b_2 \in \mathbb{R}^4,\ b_3 \in \mathbb{R}^5,\ b_4 \in \mathbb{R}^3,\ b_5 \in \mathbb{R}^1;$$
$$z_2 = w_2\phi_1 + b_2 \in \mathbb{R}^4,\ z_3 = w_3\phi_2 + b_3 \in \mathbb{R}^5,\ z_4 = w_4\phi_3 + b_4 \in \mathbb{R}^3,\ z_5 = w_5\phi_4 + b_5 \in \mathbb{R};$$
$$\phi_1 = x \in \mathbb{R},\ \phi_2 = \phi(z_2) \in \mathbb{R}^4,\ \phi_3 = \phi(z_3) \in \mathbb{R}^5 \text{ and } \phi_4 = \phi(z_4) \in \mathbb{R}^3,$$

where ϕ is the activation function.

4.2 Activation Functions

To gain some intuition why neural networks represent a suitable model for function approximation, consider the logistic sigmoid activation function which, as mentioned above, is given by:

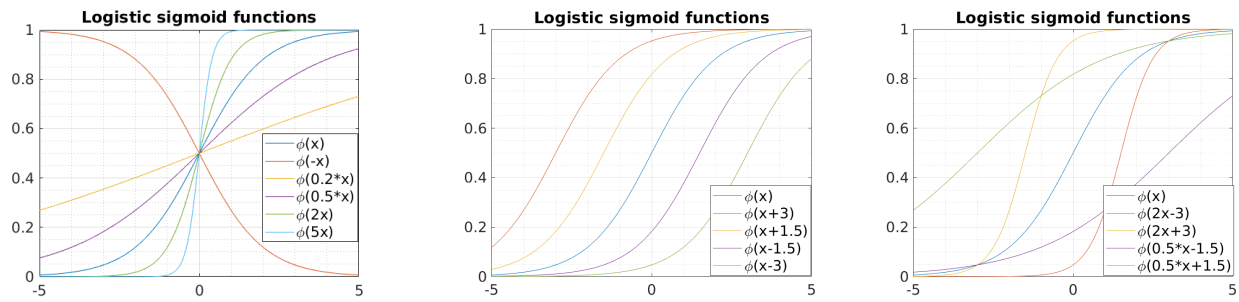
$$\phi(x) = \frac{1}{1+e^{-x}}.$$

Its graph is presented in Figure 2 for different arguments. Since the exponential function is continuous and infinitely differentiable on $\mathbb{R}$ and since the denominator does not cancel anywhere on $\mathbb{R}$, by the Algebra of Continuous Functions and the Algebra of Differentiable Functions, the logistic sigmoid is also continuous and differentiable on $\mathbb{R}$. Further, using the Chain Rule, its derivative is given by:

$$\phi'(x) = \frac{e^{-x}}{(1+e^{-x})^2} = \frac{1}{1+e^{-x}} \cdot \left(\frac{e^{-x}}{1+e^{-x}}\right) = \frac{1}{1+e^{-x}} \cdot \left(1 - \frac{1}{1+e^{-x}}\right) = \phi(x)(1-\phi(x))$$

Hence, the logistic function is also infinitely differentiable. Moreover, using the Algebra of Limits, it is easy to notice that:

$$\lim_{x\to\infty} \phi(x) = 1 \quad \text{and} \quad \lim_{x\to-\infty} \phi(x) = 0.$$



(a) (b) (c)

Figure 2: Logistic sigmoid function of different arguments

All these properties make the logistic sigmoid function a suitable candidate for constructing the approximating neural network function g. Another activation function with similar properties is the hyperbolic tangent function, $\phi(x) = \tanh(x)$.

Now, let ϕ be the logistic sigmoid and consider (3) again. In this representation, each neuron in the hidden layer represents a term in the linear combination. To see how different values of the weights and biases, change the output of the neural network function, consider Figure 2 again.

Firstly, in a neural network, the weight w_i indicates how much the ith neuron contributes to the overall output of the network. In other words, w_i represents the weighting of the ith neuron in the network. Figure 2a illustrates how different weight values in the hidden layer change the shape of the activation function. Since $\phi(x)$ is symmetric with respect to zero, $\phi(-x)$ is the flipped graph in the y-axis of $\phi(x)$. Further, the figure also shows that larger values of w_i yield a steeper curve.

As mentioned at the beginning of this section, the neurons in a neural network mimic the biological neuron. In the nervous system, a neuron fires once it passes a certain threshold. In an artificial neural network, the threshold is given by the bias parameters of the network. In other words, the bias controls the value at which activation function triggers. The effect of different values of b_i in the hidden layer is portrayed in Figure 2b. The latter further indicates that if we need to shift $\phi(x)$ to the left of zero, we require large values of b_i and, conversely, if we want to shift the function to the right of zero, we should employ smaller values of b_i.

On the one hand, in Figure 2a all the exemplified functions are centred at zero, but they have different gradients. On the other hand, in Figure 2b the functions are centred at different points, but they have the same gradient. Finally, Figure 2c illustrates the result obtained when combining different weights with different biases in the hidden layer of a shallow neural network. As expected, we obtain functions with different gradients and centred at different points, depending on the values of the parameters.

Equipped with an intuition of how neural networks can model different continuous functions, we now return to the problem of function approximation and investigate a possible approach.

4.3 Problem Formulation

Consider (3) again. As previously stated, (3) defines the function describing a shallow neural network with n neurons in the hidden layer. Then, the problem formulation can be given as follows:

Given $f : [a, b] \to \mathbb{R}$, find $g \in S$ such that $f(x) \approx g(x)$, $\forall x \in [a, b]$.

Notice that here, the space S is nonstandard, which makes the mathematical analysis more difficult. Nonetheless, we proceed as in the previous sections, and aim to answer two questions: does the function g exist, and if so, can we always find g?

Theorem 4.1. (Cybenko, 1989) Let ϕ be any continuous sigmoidal function. Then, given any $\epsilon > 0$ and $f : [a, b] \to \mathbb{R}$ such that f is continuous on $[a, b]$, there exists a sum $g(x)$, of the form:

$$g(x) = \sum_{i=1}^{n} a_i \phi_i$$

such that $\|f - g\|_\infty < \epsilon$, where $\phi_i = \phi(z_i)$, with $z_i = w_i x + b_i$.

Notice that the function g defines an artificial neural network with only one hidden layer. Then, just as Weierestrass' Theorem for polynomials, Cybenko's theorem states that there exists a function g describing a shallow artificial neural network which can approximate any continuous function arbitrarily well. However, it does not give any additional information about the number of neurons in the hidden layer or whether the function g is unique or not.

Still, based on our previous intuition, depending on the function we aim to approximate, we can find the minimum number of neurons needed in the hidden layer such that $f(x) \approx g(x)$. Consider:

$$f(x) = x^2, \quad x \in [-1, 1] \implies f'(x) = 2x, \quad x \in [-1, 1].$$

We know that the graph of f is a parabola and that f' changes sign at $x = 0$. Thus, f has an extremum at $x = 0$. As illustrated previously, one sigmoid function can render exactly one sign of

f' at a time. This further implies that we cannot use a single neuron, or equivalently, a single term in the linear combination in g to model f. In fact, we need one neuron to approximate f on $[-1, 0]$, where f' is negative, and a second one to model f on $[0, 1]$, where f' is positive.

Now, suppose $f(x) = \sin(x)$, $x \in [-2\pi, 2\pi]$. We can tell either from the graph of f, or by analytically solving $f'(x) = 0$, that f' changes sign four times on $[-2\pi, 2\pi]$. Arguing as above, we obtain that we need at least 5 neurons in the hidden layer to derive g such that $f(x) \approx g(x)$.

To generalise this result, we let f be a continuous function such that $f : [a, b] \to \mathbb{R}$. Suppose that f' exists on $[a, b]$ and that it changes its sign m times. Suppose further that g is given by:

$$g(x) = a_0 + \sum_{i=1}^{n} a_i \phi_i(x).$$

Then the minimum value of n such that $f(x) \approx g(x)$, $\forall x \in [a, b]$ is $m + 1$, which is equivalent to at least $3n + 1 = 3(m + 1) + 1 = 3m + 4$ parameters.

4.4 Approximation Algorithm - Neural Network Training

In Section 4.1, we stated that neural networks perform tasks by learning from different examples. However, we have not specified yet what 'learning' means or how it is performed in this particular context. To this purpose, we give the following definition:

Definition 4.1 (Machine Learning (Mitchell, 1997))**.** A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.

In the case of function approximation, the task T is given by the problem formulation in Section 4.3: given a continuous function $f : [a, b] \to \mathbb{R}$, find $g : [a, b] \to \mathbb{R}$ such that $f(x) \approx g(x)$, $\forall x \in [a, b]$, where g is the function representation (3) of a shallow neural network.

Further, the experience E consists in this case of the given data. For example, the function f we seek to approximate can be provided as a set of points $\{(x_i, f(x_i))\}_{i=1}^{N}$. The formula of the function f can be specified, for example $\sin(x)$ or $\ln(x)$, or not, for example the population of the United Kingdom in the last 10 years. Generally, the more data the system is trained on, the better the results it achieves.

Finally, in (1), we define $f(x) \approx g(x)$, $\forall x \in [a, b]$ to be equivalent to $f(x) = g(x) + e(x)$, $\forall x \in [a, b]$, for some small $e(x)$. In the case of function approximation, we can use the error $e(x)$ as a performance measure P. In other words, the smaller the value of $e(x)$, the better the approximation, and so, the more the neural network has learnt about the behaviour of the function f.

For a neural network to learn from an experience E, we employ a training algorithm. Generally, training neural networks implies finding appropriate values for the parameters of the network, that is, the weights and biases, such that its performance improves. In our case, the training method represents the approximation algorithm to the problem stated previously in Section 4.3.

Suppose we are given the values of a continuous function $f : [a, b] \to \mathbb{R}$ at N equidistant points in the interval $[a, b]$, which we denote by $\{(x_i, f_i := f(x_i))\}_{i=1}^{N}$. Further, let g be the function representing a shallow neural network with n neurons in the hidden layer. The parameters of the network can be organised in vector form as follows:

$$w_1 \in \mathbb{R}^{n\times 1} \text{ and } w_2 \in \mathbb{R}^{1\times n};$$
$$b_1 \in \mathbb{R}^{n} \text{ and } b_2 \in \mathbb{R}^{1}.$$

Using the definitions in Section 4.1, the output of the neural network can be defined as follows:

$$z_i = w_1 x_i + b_1, z_i \in \mathbb{R}^n, \quad a_i = \phi(z_i), a_i \in \mathbb{R}^n, \quad g_i := g(x_i) = w_2 a_i + b_2, g_i \in \mathbb{R} \tag{4}$$

where x_i is the ith training point. Let ϕ be the sigmoid activation, which acts entrywise on z_i.

Further, to simplify the notation, we denote the initial weights and biases by $\underline{w}$ and $\underline{b}$ respectively. The optimal parameters of the network, $\underline{w}^*$ and $\underline{b}^*$, are found by an iterative procedure, where they are modified to give a minimum with respect to a certain error function, $C(\underline{w}, \underline{b})$.

Since this routine is an iterative one, we need to provide an initial value for $\underline{w}$ and $\underline{b}$, for example $\underline{w} = \underline{1}$ and $\underline{b} = \underline{1}$. Then the first n entries in $\underline{w}$ represent w_1 and the following n entries represent

w_2. Similarly, the first n entries in $\underline{b}$ constitute b_1 and the last one is b_2. Next,we loop over the training examples $\{x_i\}_{i=1}^N$, compute g_i and update $\underline{w}$ and $\underline{b}$ such that the error $C(\underline{w}, \underline{b})$ is minimum.

To be able to employ the algorithm on a concrete example, we need to define $C(\underline{w}, \underline{b})$. This is known in machine learning as a cost function and it is the key component of training neural networks. We continue this section by analysing its contribution to the performance of the network.

4.4.1 Cost functions

In neural network training, cost functions provide an approach to quantifying the error between the predicted output of the network and the expected output. If we compute the error at a single training example, we obtain the corresponding loss function. Further, averaging the values of the loss function at each training point yields the cost function. Thus, the former evaluates the performance of the network at each training point, while the latter measures the overall capability of the network.

Suppose we are given the population of the United Kingdom in the past ten years and that we are aiming to find a straight line $g(x) = wx + b$ to model the given data. If we denote the value of the population in year i by f_i and the predicted value of the network at the same point by g_i, then f_i and g_i can be represented as points in the Euclidean plane. Consequently, the easiest loss function to consider is the Euclidean distance between the two points. We can then compute the value of the cost function by averaging the values of the loss function at each training example, which gives:

$$C(w, b) = \frac{1}{10} \sum_{i=1}^{10} \|f_i - g_i\|^2$$

Note that here $g(x) = wx + b$, hence the parameters w and b are real values, and not vectors. This distinct cost function is known as the Mean Squared Error and its respective loss function is:

$$C_i = \|f_i - g_i\|^2, \quad i = 1, \ldots, 10.$$

Let us return to the setting of our problem and re-write the formula for the loss function, with g defined as in the form (4), which induces the following:

$$C_i(w_1, w_2, b_1, b_2) = \|f_i - g_i\|^2 = \|f_i - (w_2 \cdot \phi(w_1 x_i + b_1) + b_2)\|^2, \quad i = 1, \ldots, N.$$

Observe that the output of the loss function depends on the values of the parameters. Therefore, the cost function, which is the average of the values of the loss function, is also a function of the weight and biases of the network. In other words, the role of this function is to map a vector of parameters to the error they produce or, equivalently, to the performance of the network.

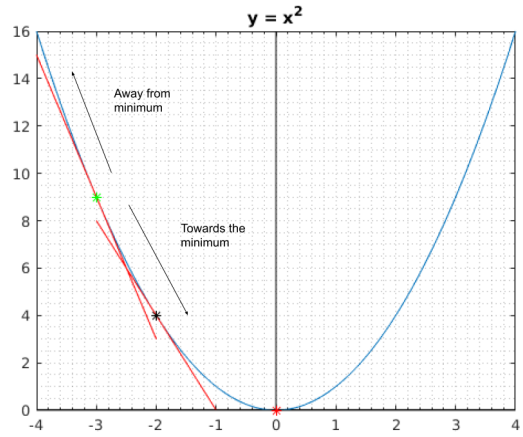
Combining the loss and cost functions with the training algorithm described in Section 4.4, we outline a possible approximation algorithm in Algorithm 1. Note that this algorithm returns the optimal parameters $(\underline{w}^*, \underline{b}^*)$. Thus, the resulting neural network function can be defined as follows:

$$\mathrm{NN} : \mathbb{R} \to \mathbb{R}, \quad \mathrm{NN}(x) := g(x, \underline{w}^*, \underline{b}^*), \quad (\underline{w}^*, \underline{b}^*) = \operatorname*{argmin}_{(\underline{w}, \underline{b})} C(\underline{w}, \underline{b})$$

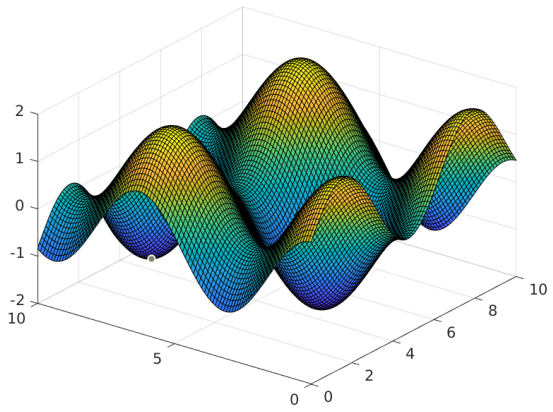
where g is defined in (4).

Training Algorithm 1:

```
Result: w*, b*
w = ones(2*n); b = ones(n+1);
for i = 0 : N do
    z = w(1:n) * x(i) + b(1:n);
    a = 1 / (1 + exp(-z));
    g(i) = w(n+1:2n) * a+ b(n+1);
    C(i) = 1 / 2 * (f(i) - g(i)) * (f(i)-g(i));
end
Cost = 1 / N * sum(C);
(w*, b*) = argmin_(w,b)Cost(w, b);
```



(a)



(b)

Figure 3

The last component of the algorithm that we must define is a procedure for minimising the cost function. This is an optimisation problem and there are many procedures which attempt to solve it.

A classical method in optimisation is the Gradient Descent method (Nocedal and Wright, 2006). The idea of this technique is to generate a sequence of vectors of parameters $(\underline{w}, \underline{b})$ with the purpose of converging to vectors $(\underline{w}^*, \underline{b}^*)$, which minimise the cost function.

To develop an intuition of how the gradient descent algorithm works, observe that the loss function is a function of the form $y(x) = x^2$. In the Cartesian coordinate system, this is the equation of a parabola, which has a global minimum at $x = 0$, as represented in Figure 3a.

To minimise the function $y = x^2$, we are required to find the optimal value of x, that is x^*, that leads to the lowest value of y. In this case, it is easy to graphically locate the global minimum of the function, which is indeed the red point $y = 0$.

Suppose that our initial guess is $x = -3$, which gives $y = 9$, represented as a green point in Figure 3a. Note that momentarily we do not know that the optimal value of x is $x^* = 0$, nor that the minimum of the function is at $y = 0$. We would like to update the value of x by a small perturbation Δx such that $y(x - \Delta x)$ approaches its minimum. Consequently, the following step is determining whether $\Delta x > 0$ or $\Delta x < 0$ and, further, how large the perturbation should be.

Firstly, to determine the sign of Δx, we can use the derivative of the function at the point x, in particular we can use $\Delta x = y'(x)$. Note that the derivative of a function at the point x defines the tangent to the graph of the function at the same point. In Figure 3a, we graph the tangent of the function $y = x^2$, whose derivative is $y'(x) = 2x$, at two different points, $(-3, 9)$ and $(-2, 4)$.

Consider the point $(-3, 9)$. It is illustrated graphically that if $\Delta x > 0$, $y(x - \Delta x)$ is further from the minimum than $y(x)$, while if $\Delta x < 0$, then $y(x - \Delta x)$ is closer to $y = 0$. In this case, $y'(-3) = -6 < 0$, thus the gradient yields indeed the right path.

The next step is establishing how large the perturbation should be. In Figure 3a, consider again the two points $(-3, 9)$, plotted in green, and $(-2, 4)$, plotted in black. The tangent at the first point is given by $y = -6x - 9$, while the tangent at the second point is given by $y = -4x - 4$. Further, it is graphically illustrated that the graph of the function at the first point is steeper than at the second point. Thus, the perturbation at $x = -2$ should be smaller than at $x = -3$. To make use of this information, we let $\Delta x = \eta \cdot y'(x)$, for x given, where η is a small positive parameter, known in practice as the learning rate of the network.

Finally, the update rule for x can be summarised in the following line:

$$x = x - \eta \cdot y'(x)$$

The rule above constitutes the Gradient Descent algorithm. In the particular case of $y(x) = x^2$, it produces a sequence of values x, converging to the optimal value x^*.

Consider again the function approximation problem and let us denote the vector of all parameters of the network, $\underline{w}$ and $\underline{b}$, by $\underline{\theta}$. Then, applying the Gradient Descent algorithm to $\underline{\theta}$ leads to:

$$\underline{\theta} = \underline{\theta} - \eta \cdot \nabla_{\underline{\theta}} C(\underline{\theta}) \tag{5}$$

where $\nabla_{\underline{\theta}} C(\underline{\theta})$ is the gradient of C with respect to $\underline{\theta}$.

Generally, the algorithm runs or, equivalently, the parameters are updated, until a certain condition is met, which is also known as the termination condition. Commonly, the algorithm stops either when the improvement in the parameters θ drops under a previously chosen threshold, known as tolerance, or when the maximum number of iterations has been reached.

In practice, the Gradient Descent algorithm has various disadvantages. First of all, consider the simplest shallow neural network with only one neuron in the hidden layer. In the function approximation problem, the cost function is, in this case, a function of $3 \times 1 + 1 = 4$ variables. Finding the optimal parameters $\underline{\theta}^*$ is equivalent to finding a global minimum of the cost function. This is a convoluted task, especially in higher dimensions and frequently, the algorithm returns the value of $\underline{\theta}$ which corresponds to a local minimum. To illustrate this, in Figure 3b we plot a possible cost function. As portrayed in the graph, this function has more than one minimum. Once the algorithm starts 'descending' towards the local minimum, since it has no knowledge about the existence of the other local minima, it continues until the termination condition is met or the maximum number of iterations has been reached.

Furthermore, η and the maximum number of iterations are known as hyperparameters, signifying that their values are set before the training process starts and are not modified during training. It is usually difficult to determine their value and this is done experimentally.

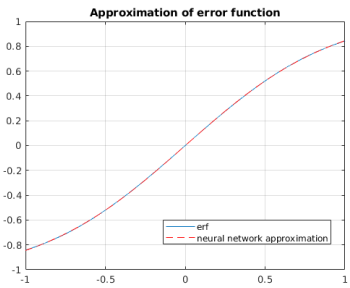
There are a few variations of the Gradient Descent algorithm (Ruder, 2016). The most recent and used algorithm in training deep neural networks is the Adam algorithm (Kingma and Ba, 2014). This is also one of the fastest gradient descent optimisation algorithms because it is based on the idea of adaptive learning. In other words, the Adam algorithm changes the size of the update value adaptively during learning, contrary to having a fixed learning rate η.

4.5 Numerical experiments

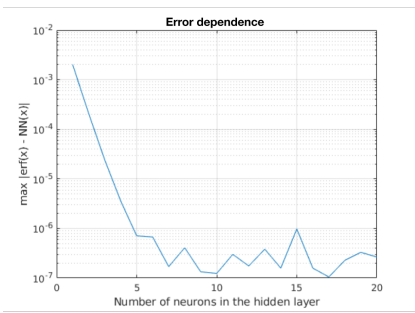
To illustrate how Algorithm 1 works, consider the following functions:

$$\mathrm{erf}(x) = \frac{1}{2\sqrt{\pi}} \int_0^x e^{-t^2} dt, \quad x \in [-1, 1] \quad \text{and} \quad f(x) = \frac{1}{1 + x^2}, \quad x \in [-5, 5]. \tag{6}$$

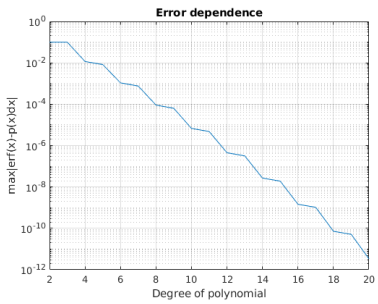
The first function in (6) is known as the error function and it exemplifies a clear application of function approximation, as the integral in the definition of the function cannot be computed explicitly. Next, suppose the interval $[-1, 1]$ is divided in 10 equidistant subintervals and suppose further that we are given the values of the function at each of the partitioning points. Accordingly, the training data is $\{(x_i, f_i := \mathrm{erf}(x_i))\}_{i=0}^{10}$. Moreover, in Figure 4a we plot the graph of the error function. As illustrated, the function is strictly increasing on $[-1, 1]$, which implies that one neuron in the hidden layer of a shallow neural network yields a reasonably good approximation.



(a) Neural Network Approximation



(b) Error dependence on n



(c) Error dependence on m

Figure 4: Error function approximation

Note that due to its disadvantages, we opted for a different optimisation routine than Gradient Descent in our training algorithm. In particular, experiments and research suggested that the Levenberg-Marquardt method (Moré, 2006) leads to the best results. We implemented Algorithm 1 in MATLAB and employed the `lsqnonlin` solver avaiable in MATLAB's Optimisation Toolbox, selecting the Levenberg-Marquadt procedure mentioned above.

In Figure 4a, we present the approximation obtained using a shallow neural network with one neuron in the hidden layer, that is, with 4 parameters. Further, in Figure 4b, we display the correlation between the number of neurons in the hidden layer n and the L^∞ error between the error function and its approximation. In other words, we graph:

$$\max_{x\in[-1,1]} |\operatorname{erf}(x) - \mathrm{NN}(x)|, \quad \text{for } n = 1, \ldots, 20.$$

The rapid decay from $n = 1$ to $n = 5$ in Figure 4b suggests that the error decreases exponentially. However, due to the limitations of the solver, if the number of neurons in the hidden layer is larger than 5, the error fluctuates between 10^{-6} and 10^{-7} and it does not diminish further.

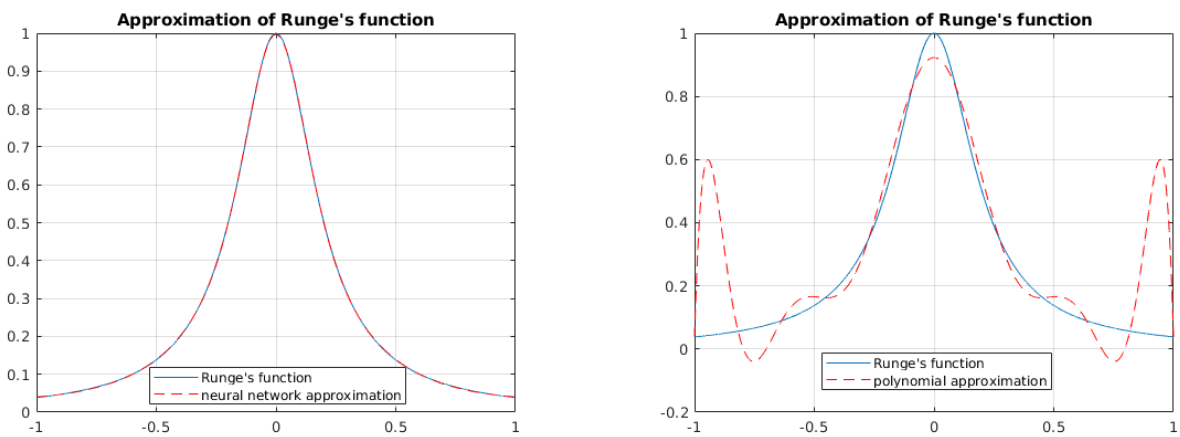
For comparison, we additionally implemented a polynomial approximation algorithm. In Figure 4c, we highlight the correspondence between the degree of the polynomial $p \in \Pi_m$, with $m + 1$ parameters, and the L^∞ error between the error function and its polynomial approximation. In contrast, using 4 parameters, that is, using a polynomial $p \in \Pi_3$, the error is roughly 10^{-1}. This is notably larger than approximately 10^{-3}, which we obtained with the same number of parameters in the neural network procedure.

However, if we analyse the graph further, we notice that if we require higher precision, the polynomial approximation is superior in terms of computation. For example, if we consider $n = 5$ neurons in the hidden layer, that is 16 parameters, we obtain an error of approximately 10^{-6}, while the polynomial approach with the same number of parameters, that is $p \in \Pi_{15}$, yields and error of roughly 10^{-8}. Consequently, in practice, choosing the appropriate technique is relative to the purpose of the application.

Next, let us consider the second function in (6), known as Runge's function. Note that this function is continuous and infinitely differentiable on $\mathbb{R}$. Observe further that approximating Runge's function for $x \in [-1, 1]$ is equivalent to approximating:

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1].$$

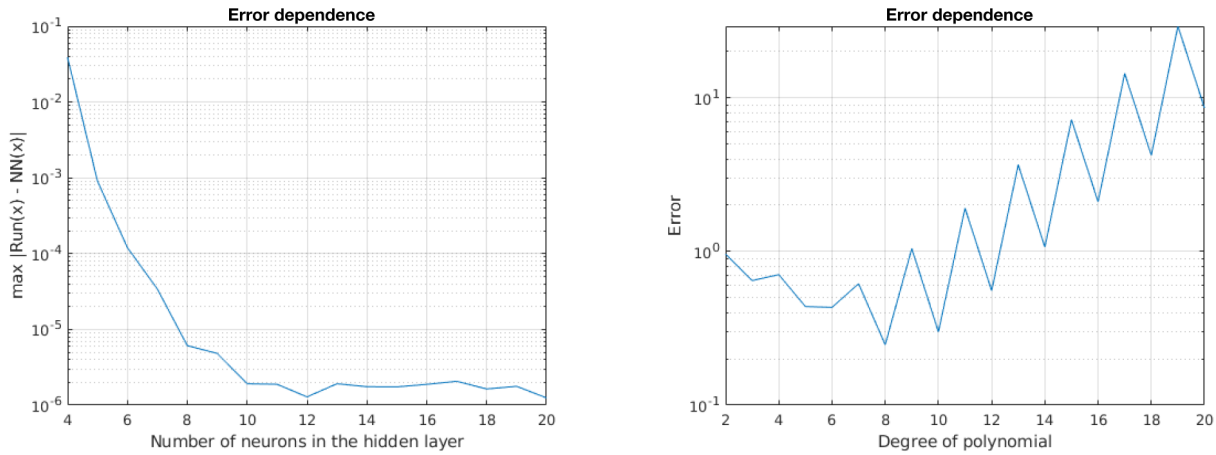
In Figure 5a, we plot the function above for $x \in [-1, 1]$. Similarly as with the error function example, suppose the interval $[-1, 1]$ is divided in 10 equal subintervals. We can compute the values of the function at each of the partitioning points, so that the training data is $\{(x_i, f_i := f(x_i))\}_{i=0}^{10}$.



(a) Neural Network Approximation

(b) Polynomial Approximation

Figure 5: Runge's Function Approximation



(a) Error dependence on n

(b) Error dependence on m

Figure 6: Polynomial interpolation on equidistant nodes of Runge's function

If we select $n = 4$ neurons in the hidden layer and train the neural network on the given data, we obtain the approximation illustrated in Figure 5a.

Conversely, consider the polynomial approximation algorithm that we implemented for the error function. If we apply it to Runge's function on equidistant nodes, we obtain the result portrayed in Figure 5b. This is known as Runge's phenomenon, which establishes, as Süli and Mayers (2003) state, that for Runge's function, the L^∞ error between $f(x)$ and $p(x) \in \Pi_m$ increases exponentially as m increases. This is additionally illustrated in Figure 6b, for $m = 2, \ldots, 20$.

In Figure 6a, we plot the L^∞ error on $[-1, 1]$ between Runge's function and $\mathrm{NN}(x)$, the function obtained after training the neural network on the given data. Even though the decline is not as fast as in Figure 4b, the error decreases nonetheless from approximately 10^{-1} to 10^{-6}. It does not reduce further due to the limitations of the `lsqnonlin` solver. Consequently, this emphasises that neural networks do not suffer from Runge's phenomenon.

Another impressive numerical experiment is approximating the exponential function, $f(x) = e^x$. Consider again the interval $[-1, 1]$ split uniformly in 4 subintervals, so that we have 5 partitioning points. Accordingly, we denote the training data by $\{(x_i, f_i := e^{x_i})\}_{i=0}^4$. Employing a shallow neural network with exactly one neuron in the hidden layer and using no more than five training points, we obtain an L^∞ error of 6.2453×10^{-7}. Similarly as before, we cannot further analyse the decline in the maximum error on the interval $[-1, 1]$ due to the limitations of the MATLAB's `lsqnonlin` solver. That is, if we plot the graph which illustrates the correlation between the L^∞ error and the number of neurons in the hidden layer n, the maximum error does not reduce. On the contrary, it fluctuates between 10^{-6} and 10^{-7}. Nevertheless, the initial error we obtained with 4 parameters and 5 training points remains noteworthy.

The purpose of the numerical experiments is to find a connection between the number of neurons in the hidden layer n and the L^∞ error between the given function f and the obtained approximation $\mathrm{NN}(x)$. From this perspective, we introduce an apparently simple, but compelling experiment.

Consider intervals of the form $[-2^a, 2^a]$ of length 2^{a+1}, $a \in \mathbb{Z}$. Suppose the intervals are divided uniformly in N subintervals, leading to $N+1$ partitioning points. Suppose further that $f(x) = p_m(x)$, $p_m(x) \in \Pi_m$. We denote the training data accordingly by $\{(x_i, f_i := p_m(x_i) = x_i^m)\}_{i=0}^N$, $m \in \mathbb{N}$. Further, we train a shallow neural network on the given data, using the same algorithm as in the previously presented experiments. The results are presented in Table 4.5.

2^{a+1}	2^5	2^4	2^3	2^2	2^1	2^0	2^{-1}
$m = 1$	$2.8 \cdot 10^{-4}$	$1.4 \cdot 10^{-4}$	$7.1 \cdot 10^{-5}$	$3.3 \cdot 10^{-5}$	$1.7 \cdot 10^{-5}$	$8.4 \cdot 10^{-6}$	$4.2 \cdot 10^{-6}$
$m = 2$	$5 \cdot 10^{-3}$	$1.1 \cdot 10^{-3}$	$2.3 \cdot 10^{-4}$	$5.8 \cdot 10^{-5}$	$1.45 \cdot 10^{-5}$	$4 \cdot 10^{-6}$	$1.1 \cdot 10^{-6}$
$m = 3$		$9 \cdot 10^{-3}$	$1.1 \cdot 10^{-3}$	$1.4 \cdot 10^{-4}$	$1.7 \cdot 10^{-5}$	$2.5 \cdot 10^{-6}$	$3.2 \cdot 10^{-7}$

The size of the intervals considered $[-2^a, 2^a]$, that is 2^{a+1}, is given in the first row of Table 4.5. In the following three rows, we display the L^∞ error between $\mathrm{NN}(x)$ and $p_m(x) \in \Pi_m$ for different values of m, on the corresponding interval or, equivalently:

$$\max_{x\in[-2^a,2^a]} |\,\mathrm{NN}(x) - p_m(x)|, \quad a \in \mathbb{Z}, \quad p_m \in \Pi_m.$$

First, consider the $m = 1$ case. Since $p_1(x) = x$ is linear, $n = 1$ neuron in the hidden layer of the network suffices to obtain an adequate approximation $\mathrm{NN}(x)$. In this instance, observe that, as the size of the interval decreases by a factor 2, the maximum error between x and its approximation also reduces roughly by a factor of 2^1. In other words, the investigated error appears to drop linearly with the size of the approximation interval.

Next, we examine the $m = 2$ example, which is a quadratic polynomial. Consequently, we choose $n = 2$ neurons in the hidden layer of the network to generate the approximation $\mathrm{NN}(x)$ of $p_2(x) = x^2$. In this case, note that the L^∞ error between $p_2(x)$ and $\mathrm{NN}(x)$ diminishes by a factor of 2^2 when the size of the interval shrinks by a factor of 2. Equivalently, the error decreases quadratically with the size of the approximation interval.

Finally, because x^3 increases very quickly, we cannot obtain a reasonable approximation $\mathrm{NN}(x)$ with $n = 3$ neurons in the hidden layer on a large interval, such as $[-2^4, 2^4]$. Nevertheless, if we consider the other cases, we observe that the maximum error between x^3 and its approximation lessens by a factor of about 2^3 as the approximation interval narrows by a factor of 2. Correspondingly, the error appears to drop in a cubic manner with the size of the approximation interval.

For $m \geqslant 4$, the factor by which the error decreases as the interval halves is at least 2^4. As the maximum error drops very rapidly, due to the limitations of the solver we employed, the result becomes increasingly more difficult to illustrate.

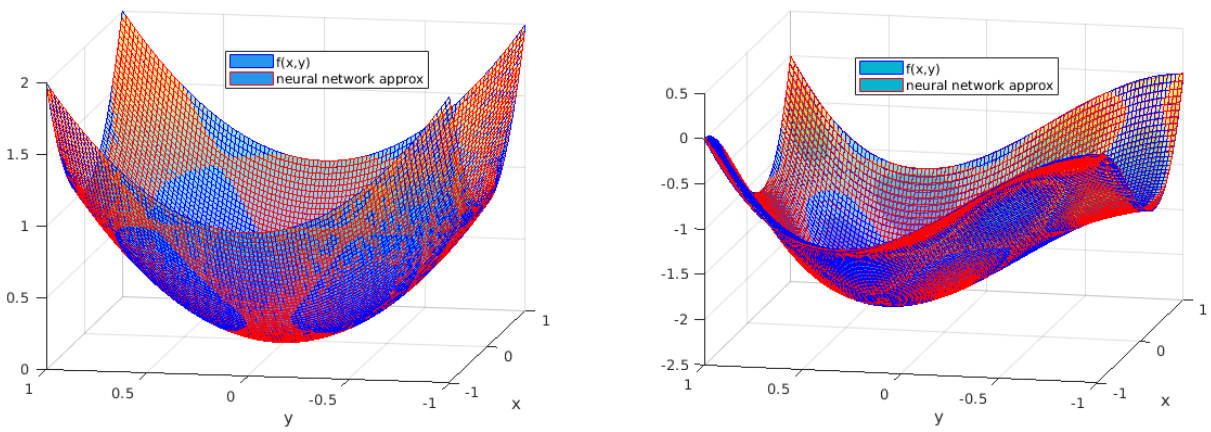
Additionally, Barron (1993) offers a theoretical result which relates the number of neurons in the hidden layer of a shallow neural network with the error between the given function and its approximation. Particularly, the article imposes a bound on the integrated squared error, defined as:

$$C(w, b) = \int_a^b (\mathrm{NN}(x) - f(x))^2 dx.$$

Here, $\mathrm{NN}(x)$ represents a neural network with one hidden layer, applying a sigmoid function as the activation of the network. The paper states that for certain functions f, the neural network NN achieves an error of order $\mathcal{O}(1/n)$, where n is the number of neurons in the hidden layer.

5 Discussion

One of the important merits of neural networks is their potential to work in multiple dimensions. In contrast with other methods, for example polynomial approximation, here the function $\mathrm{NN}(x)$



(a) (b)

Figure 7: Functions of two variables shallow neural network approximation

hardly changes. Consider a function of d variables, $f : \mathbb{R}^d \to \mathbb{R}$. In the former case, constructing the approximating polynomial $p : \mathbb{R}^d \to \mathbb{R}$ becomes rapidly more complex as $d \geqslant 2$. On the other hand, to obtain a shallow neural network approximation $\mathrm{NN} : \mathbb{R}^d \to \mathbb{R}$, we require:

$$w_1 \in \mathbb{R}^{n\times d} \text{ and } w_2 \in \mathbb{R}^{1\times n};$$
$$b_1 \in \mathbb{R}^d \text{ and } b_2 \in \mathbb{R}^1,$$

while z, a and g are defined in (4). Consequently, the fundamental change as d increases is the number of parameters of the network. All the other components in the training algorithm remain consistent. As an illustration, in Figure 7a, we plot $f(x, y) = x^2 + y^2$ and its approximation employing a shallow neural network with $n = 4$ neurons in the hidden layer. Further, in Figure 7b, we graph $f(x, y) = x^3 + x^2 - x + y^3 + y^2 - y - 2$ and the corresponding approximation with $n = 4$.

Occasionally, in spite of their distinguished approximation properties, neural networks exhibit unpromising behaviour. Observe that all the functions considered in the numerical experiments in Section 4.5 are not only continuous, but also differentiable. Consider now $f : \mathbb{R} \to \mathbb{R}$, given by:

$$f(x) = \ln\left(|x| + \frac{1}{e}\right) \tag{7}$$

Note that:

$$\lim_{h\to 0^+} \frac{f(h) - f(0)}{h} = \lim_{h\to 0^+} \frac{\ln(h + e^{-1}) + 1}{h} = -\infty$$

and further that:

$$\lim_{h\to 0^-} \frac{f(h) - f(0)}{h} = \lim_{h\to 0^-} \frac{\ln(-h + e^{-1}) + 1}{h} = \infty$$

Thus, $f(x)$ is not differentiable at $x = 0$. The maximum L^∞ error we achieved when employing a shallow neural network with different values of n is 0.2, which is strikingly larger compared to the previous experiments. Accordingly, the maximum error occurs at the point $x = 0$.

Even though shallow neural networks fail to converge to the given function $f : [a, b] \to \mathbb{R}$, if f is not differentiable at some point $x \in [a, b]$, we can employ a different approach where differentiablity of the function f is not an issue.

Suppose we wish to approximate $f : [a, b] \to \mathbb{R}$. Suppose further that the interval $[a, b]$ is split into m subintervals $[x_i, x_{i+1}]$, $i = 0, \ldots, m - 1$, with $x_0 = a$ and $x_m = b$. Instead of constructing the approximation $\mathrm{NN} : [a, b] \to \mathbb{R}$ of f using the partition points as input data, we can consider each subinterval separately and train the neural network on each $[x_i, x_{i+1}]$. Assume that we are given N training points on each subinterval, including the endpoints, where N is a small number such as $N = 3$ or $N = 4$. Accordingly, we denote the training set for the ith subinterval by $\{(x_{i,j}, f(x_{i,j})\}_{j=0}^N$, $i = 0, \ldots, m - 1$. Observe that this yields a continuous function, since:

$$\mathrm{NN}(x_{i,N}) = \mathrm{NN}(x_{i+1,1}), \quad \forall\, i = 0, \ldots, m - 1.$$

This method is known as piecewise approximation and it is also employed with polynomials. Further, there are multiple advantages with this approach. Consider example 7 again. In this case, if the interval of approximation is split appropriately, that is, if we let $x = 0$ be one of the partitioning points, then $\|\,\mathrm{NN} - f\|_\infty$ decreases up to $10^{-5} << 0.2$, which is the error we obtained originally. The immediate question is why we don't obtain a smaller error in example 7, if we let $x = 0$ be one of the training points. Recall that the logistic sigmoid function is a smooth function, that is infinitely many times differentiable. Since NN is a linear combination of sigmoids, NN is also a smooth function. However, the function f is not smooth on $[a, b]$ with $0 \in [a, b]$. Consequently, NN cannot generate the behaviour of f on the approximation interval, even if $x = 0$ is a training point. On the other hand, f is piecewise smooth, thus the piecewise approximation yields a better result.

Finally, as mentioned in Section 4.4, the variables $\underline{w}$ and $\underline{b}$ need to be initialised. In our algorithm, we used $\underline{w} = \underline{1}$ and $\underline{b} = \underline{1}$, which is especially useful when analysing how other hyperparameters influence the resulting approximation. However, if the initial parameters are sampled from a normal distribution, we notice that the initial guess has a great impact on the quality of the approximation. This is expected, since different initial points might be closer to different local minima, but nonetheless problematic.

6 Application

Some applications of function approximation have already been mentioned. One of the most important ones is data modelling. In other words, given a data set, such as air temperature, population of a country, or stock markets, using function approximation, we can model the underlying trend and predict possible future values. A second important example is approximating functions which cannot be evaluated using arithmetic operations directly, such as the error function, $\ln(x)$, or $\sin(x)$.

Another application of function approximation is solving boundary value problems. Recall Definition 4.1. Suppose the experience E, or, equivalently, the training data is given implicitly, as follows:

$$-\frac{d^2u}{dx^2} = f, \quad x \in (a, b)$$
$$u(x) = 0, \quad x \in \{a, b\}$$

In this case, $u : [a, b] \to \mathbb{R}$ is a one-variable function. If we let u be a function of 2 variables, we obtain what is known as the Poisson's equation, with important applications in mathematics.

Furthermore, we can apply a similar method as we did in Section 4.4 to achieve the solution to the boundary value problem above. Note that if we do not impose the boundary conditions, the solution to the differential equations is given by a family of functions. Consequently a neural network algorithm could yield any function NN which satisfies the differential equation. Thus, the boundary conditions are essential to derive a particular solution.

The neural network procedure has the potential to work on boundary value problems in multiple dimensions (E and Yu, 2018). This is extremely important as generally, in practice, current methods fail on functions of more than 2 variables.

7 Conclusion

As stated at the beginning, this report introduces a novel method for approximating functions. We briefly describe some well-known procedures in order to emphasise the potential of the neural network approach. We first formulate the problem, then propose an approximation algorithm and finally present numerical experiments. We highlight advantages and disadvantages of this technique, as well as possible applications.

References

Atkinson, K. E. (1989). *An Introduction to Numerical Analysis*. John Wiley & Sons.

Barron, A. R. (1993). "Universal approximation bounds for superpositions of a sigmoidal function". In: *IEEE Transactions on Information Theory* 39.3, pp. 930–945. DOI: 10.1109/18.256500.

Cheney, E. W. (1966). *Introduction to Approximation Theory*. McGraw-Hill.

Cybenko, G. (1989). "Approximation by Superpositions of a Sigmoidal Function". In: *Mathematics of Control Signals and Systems* 2, pp. 303–314. DOI: https://doi.org/10.1007/BF02551274.

E, W. and B. Yu (2018). "The Deep Ritz Method [A Deep Learning-Based Numerical Algorithm for Solving Variational Problems]". In: *Communications in Mathematics and Statistics* 6.1. DOI: https://doi.org/10.1007/s40304-018-0127-z.

Kingma, D. P. and J. Ba (2014). "Adam: A Method for Stochastic Optimization". In: arXiv: 1412.6980.

Mitchell, T. M. (1997). *Machine Learning*. 1st ed. McGraw-Hill, Inc.

Moré, J. J. (2006). "The Levenberg-Marquardt algorithm: Implementation and theory". In: *Watson G.A. (eds) Numerical Analysis. Lecture Notes in Mathematics* 630, pp. 105–116. DOI: https://doi.org/10.1007/BFb0067700.

Nocedal, J. and S. J. Wright (2006). *Numerical Optimization*. 2nd edn. Springer.

Ruder, S. (2016). "An overview of gradient descent optimization algorithms". In: arXiv:1609.04747.

Süli, E. and D. F. Mayers (2003). *An Introduction to Numerical Analysis*. Cambridge University Press.